

Introduction to AI Agents

AI That Plans, Acts & Achieves Goals

■ Day	22
■ Duration	6 Hours
■ Difficulty	Easy → Intermediate
■ Language	Python 3.10+
■ Framework	LangChain · OpenAI · Wikipedia API

By the end of this guide you will understand how AI Agents work,
build your own agent with real tools, and crack interview questions with confidence.

Table of Contents

1	Chatbot vs AI Agent — What's the Real Difference?
2	How Agents Think — The ReAct Loop
3	Agent Building Blocks — Brain, Tools, Memory, Goal
4	Setting Up Your Environment
5	Your First Agent — Compound Interest Calculator
6	Lab — Wikipedia Research Agent
7	Real-World Agent Examples
8	Common Errors & Debugging Tips
9	Interview Questions & Model Answers
10	Quick Revision Cheatsheet

Section 1 · Chatbot vs AI Agent — What's the Real Difference?

Most people confuse a chatbot with an AI agent. The difference is enormous — and understanding it is the foundation of everything in this chapter.

The Simple Analogy

Imagine asking two people to book you a flight to Goa for next Friday.

Feature	Chatbot ■	AI Agent ■
What it does	Answers your question	Takes action on your behalf
Planning	None — one-shot reply	Multi-step planning
Tools	None	Search, APIs, code, browser
Memory	Usually session only	Can persist across steps
Example	"Flights to Goa cost ~■4,500"	Books the cheapest flight & emails you
Output	Text response	Real-world action / result

Real-World Agent Examples

Here are agents that exist today doing real tasks autonomously:

✈ ■ Travel Booking Agent	Searches flights, compares prices, fills booking form, sends confirmation email.
■ Email Agent	Reads your inbox, drafts replies, schedules meetings in Google Calendar.
■ Data Analysis Agent	Downloads a CSV, runs Python analysis, plots charts, writes a summary report.
■ Web Browsing Agent	Opens websites, fills forms, clicks buttons, extracts structured data.
■ Coding Agent (Claude Code)	Reads your codebase, writes new features, runs tests, fixes bugs autonomously.

Section 2 · How Agents Think — The ReAct Loop

ReAct stands for Reason + Act. It is the core thinking pattern used by almost every modern AI agent. The agent loops through three stages until it reaches the final answer.

The Three Stages

■ REASON

The LLM looks at the current situation and thinks: "What information do I have? What do I still need? What is the best next action?"

■ ACT

The agent calls a tool — search the web, run a calculator, read a file, call an API, write to a database. The tool executes and returns a result.

■ OBSERVE

The agent reads the tool's output and adds it to its context. It now knows more than before and re-evaluates: is the goal achieved?

Trace of a Real ReAct Loop

Task given to the agent: "What is the population of Tokyo and how does it compare to Mumbai?"

Step	Stage	What happens
1	REASON	I need Tokyo's population. I'll search for it.
2	ACT	search("Tokyo population 2024")
3	OBSERVE	Result: Tokyo population ≈ 13.96 million (city proper)
4	REASON	Now I need Mumbai's population too.
5	ACT	search("Mumbai population 2024")
6	OBSERVE	Result: Mumbai population ≈ 20.7 million
7	REASON	I have both numbers. Mumbai is larger. I can now answer.
8	FINAL	Tokyo (13.96M) vs Mumbai (20.7M) — Mumbai is ~48% larger.

■ *Key insight: The agent doesn't guess. It uses tools to find real answers, just like a smart human researcher would.*

Section 3 · Agent Building Blocks — Brain, Tools, Memory, Goal

Every AI agent, no matter how complex, is built from four fundamental components. Master these and you can build any agent.

● ■ The Brain (LLM)

- The large language model — GPT-4, Claude, Mistral, Llama — does all the thinking.
- It reads instructions, reasons about what to do next, chooses tools, and writes the final answer.
- Popular choices: GPT-4o (OpenAI), Claude Sonnet (Anthropic), Mistral-7B (open source).
- The LLM itself has no memory, no internet, no code execution — that's what tools are for.

● ■ Tools

- Tools are functions the agent can call to interact with the world.
- Examples: `web_search()`, `calculator()`, `read_file()`, `send_email()`, `run_python()`.
- Each tool has a name, description (so the LLM knows when to use it), and input/output schema.
- You can write custom tools for anything: database queries, Slack messages, IoT devices.

● ■ Memory

- Short-term: the full conversation / action history in the context window (4K–200K tokens).
- Long-term: vector databases (Pinecone, ChromaDB) that store past conversations as embeddings.
- Entity memory: key facts extracted and stored separately — names, dates, preferences.
- Without memory, every agent call starts from scratch — like waking up with amnesia.

● ■ Goal

- The task the agent is trying to complete — given by the user in the system prompt.
- Can be simple: 'Answer this question' or complex: 'Book cheapest flight to Delhi next weekend'.
- The agent loops through Reason → Act → Observe until the goal is achieved or it gives up.
- You can also give agents sub-goals and let them decompose tasks automatically.

Section 4 · Setting Up Your Environment

Step 1 — Install Required Libraries

Open your terminal (Command Prompt / Terminal on Mac/Linux) and run:

```
■ bash

# Install LangChain and OpenAI integration
pip install langchain langchain-openai langchain-community

# Install Wikipedia tool
pip install wikipedia

# Install dotenv for API key management (best practice)
pip install python-dotenv

# Verify installation
python -c "import langchain; print(langchain.__version__)"
```

Step 2 — Get Your OpenAI API Key

1. Go to platform.openai.com → Sign Up / Log In 2. Click top-right avatar → 'API Keys' → 'Create new secret key' 3. Copy the key — it starts with sk-... 4. Store it in a .env file (NEVER hardcode in your Python script!)

```
■ bash

# .env file – create this in your project folder
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxx

# .gitignore – add this line so you don't accidentally push your key!
.env
```

Step 3 — Test Your Setup

```
■ python

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
import os

# Load API key from .env file
load_dotenv()

# Initialize the LLM (the 'brain' of our agent)
llm = ChatOpenAI(
    model='gpt-4o-mini', # cheaper model, great for learning
    temperature=0, # 0 = deterministic, consistent answers
    api_key=os.getenv('OPENAI_API_KEY')
)

# Quick test – should print a response
response = llm.invoke('Say hello in one sentence')
print(response.content)

# Expected: Hello! How can I assist you today?
```

■■ **Note:** GPT-4o-mini costs ~\$0.15 per million input tokens. The labs in this guide cost less than ■5 total. If you don't have an OpenAI key, see the free alternatives section at the end.

If you ask a plain LLM 'What is the compound interest on £50,000 at 8% for 5 years?' it might hallucinate the

```
python
# compound_agent.py
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain import hub
import math

load_dotenv()

# TOOL DEFINITION
def compound_interest(query: str) -> str:
    """
    Calculates compound interest.
    Input format: 'principal,rate,years'
    Example: '50000,8,5'
    Returns: Final amount and interest earned.
    """
    try:
        parts = query.replace(' ', '').split(',')
        principal = float(parts[0])
        rate = float(parts[1]) / 100 # convert % to decimal
        years = float(parts[2])

        # Formula: A = P * (1 + r)^n
        final_amount = principal * math.pow(1 + rate, years)
        interest = final_amount - principal

        return (
            f'Principal: Rs.{principal:,.2f}\n'
            f'Rate: {rate*100}% per year\n'
            f'Time: {years} years\n'
            f'Final Amount: Rs.{final_amount:,.2f}\n'
            f'Interest Earned: Rs.{interest:,.2f}'
        )
    except Exception as e:
        return f'Error: {str(e)}. Use format: principal,rate,years'

# REGISTER TOOL
```

[illegible]

Expected Output (verbose=True shows the ReAct loop):

```
■ text
> Entering new AgentExecutor chain...

Thought: I need to calculate compound interest.
I have a calculator tool for this.
Action: CompoundInterestCalculator
Action Input: 50000,8,5

Observation:

Principal: Rs.50,000.00
Rate: 8.0% per year
Time: 5.0 years
Final Amount: Rs.73,466.40
Interest Earned: Rs.23,466.40

Thought: I now have all the information needed to answer.
```


Final Answer: If you invest Rs.50,000 at 8% compound interest per year for 5 years, your investment grows to Rs.73,466.40. The total interest earned is Rs.23,466.40.

> Finished chain.

Understanding Each Line of Code

`create_react_agent()`

Builds the agent by wiring the LLM + tools + ReAct prompt together. The prompt tells the LLM exactly HOW to reason and call tools.

`AgentExecutor()`

The runtime engine that actually runs the Reason→Act→Observe loop. `max_iterations` prevents infinite loops if the agent gets confused.

`Tool(name, func, description)`

Wraps your Python function as an agent tool. The description is crucial — the LLM reads it to decide WHEN to call this tool.

`verbose=True`

Prints every thought, action, and observation. Always use this while learning — it's like watching the agent's inner monologue.

`hub.pull('hwchase17/react')`

Downloads the standard ReAct prompt that teaches the LLM to output Thought/Action/Action Input/Observation in the correct format.

Section 6 · Lab — Wikipedia Research Agent

Now we build a more powerful agent that can search Wikipedia to answer any factual question. This demonstrates chaining multiple tool calls.

The Full Wikipedia Agent

[illegible]

[illegible]

Example 1 — Custom Tool: Get Weather

Here's how to write a tool that calls a real API. This pattern works for ANY API — weather, stocks, databases, Slack, etc.

■ python

```
import requests
from langchain.tools import Tool

def get_weather(city: str) -> str:
    """Gets current weather for a city using Open-Meteo (free, no API key)."""
    try:
        # First, geocode city to lat/lon
        geo_url = f'https://geocoding-api.open-meteo.com/v1/search?name={city}&count;=1'
        geo_resp = requests.get(geo_url).json()

        if not geo_resp.get('results'):
            return f'City not found: {city}'

        result = geo_resp['results'][0]
        lat, lon = result['latitude'], result['longitude']
        city_name = result['name']

        # Get weather
        wx_url = (
            f'https://api.open-meteo.com/v1/forecast?'
            f'latitude={lat}&longitude;={lon}'
            f'&current;=temperature_2m,weathercode,windspeed_10m'
        )
        wx = requests.get(wx_url).json()['current']

        return (
            f'Weather in {city_name}: \n'
            f'Temperature: {wx["temperature_2m"]}°C \n'
            f'Wind Speed: {wx["windspeed_10m"]} km/h'
        )
    except Exception as e:
        return f'Error fetching weather: {e}'

weather_tool = Tool(
    name='WeatherChecker',
    func=get_weather,
    description='Get current weather for any city. Input: city name. Example: Mumbai'
)
```

Example 2 — Memory: Remembering Past Answers

■ python

```
from langchain.memory import ConversationBufferMemory
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

# Create memory that stores conversation history
```

```

memory = ConversationBufferMemory(
    memory_key='chat_history',
    return_messages=True
)

# Use CONVERSATIONAL ReAct prompt (supports memory)
prompt = hub.pull('hwchase17/react-chat')
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True
)

# First question
agent_executor.invoke({'input': 'My name is Priya and I am learning AI.'})

# Agent now remembers – even in next message!
result = agent_executor.invoke({'input': 'What is my name?'})
print(result['output']) # Output: Your name is Priya.

```

Section 8 · Common Errors & Debugging Tips

Error	Cause	Fix
AuthenticationError: Invalid API key	Your OPENAI_API_KEY is wrong or missing.	Check .env file. Make sure you called load_dotenv() BEFORE.
OutputParserException	LLM returned text in wrong format — missed <code>Thought/Action pattern</code> .	Add <code>handle_parsing_errors=True</code> to AgentExecutor. Use <code>regex_parser</code> .
RateLimitError	You've hit OpenAI's requests-per-minute limit.	Add <code>time.sleep(2)</code> between calls. Upgrade plan or use gpt-4o.
ModuleNotFoundError: langchain_openai	Package not installed or wrong name.	pip install langchain-openai (note the dash, not underscore)
Agent loops forever / max_iterations hit	Agent can't figure out how to use the tool correctly.	Improve the tool description. Check that tool returns useful results.
wikipedia.exceptions.PageError	Wikipedia page not found for the search term.	Use <code>wikipedia.search()</code> first to get valid page titles.

Pro Debugging Workflow

■ python

```
# Set verbose=True to see every step
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Enable LangChain's debug mode for even more detail
import langchain
langchain.debug = True

# Print the exact prompt being sent to the LLM
from langchain.callbacks import StdOutCallbackHandler
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    callbacks=[StdOutCallbackHandler()]
)

# Inspect tool outputs manually before adding to agent
print(search_wikipedia('quantum computing')) # test your tool directly!
print(compound_interest('50000,8,5')) # always test tools first
```

Section 9 · Interview Questions & Model Answers

These are real interview questions asked at top companies (Google, Microsoft, AI startups). Study these carefully — they cover both concepts and code.

Q1. What is an AI Agent? How is it different from a chatbot?

An AI Agent is a system that perceives its environment, makes decisions, uses tools, and takes actions autonomously to achieve a goal — it loops until the task is done. A chatbot responds to a single message with text. Key difference: an agent ACTS in the world; a chatbot only SPEAKS.

Q2. Explain the ReAct framework.

ReAct = Reason + Act. The agent alternates between (1) Reasoning: thinking about what to do next, (2) Acting: calling a tool, and (3) Observing: reading the tool's output. This loop continues until the goal is achieved. It was introduced in the paper 'ReAct: Synergizing Reasoning and Acting in LLMs' (2022).

Q3. What are the four main components of an AI Agent?

(1) Brain/LLM: the model that does reasoning (GPT-4, Claude, Mistral). (2) Tools: functions the agent can call (search, calculator, APIs). (3) Memory: short-term (context window) and long-term (vector DB). (4) Goal: the task given by the user that the agent tries to complete.

Q4. What is a 'Tool' in LangChain and how do you create one?

A Tool is a Python function wrapped with metadata (name, description) that tells the LLM when and how to call it. You create one using the Tool() class: `Tool(name='WeatherSearch', func=get_weather, description='Gets weather for a city.')`. The description is read by the LLM to decide when to invoke the tool.

Q5. What is the purpose of max_iterations in AgentExecutor?

It's a safety guardrail. If the agent gets stuck in a loop (e.g., keeps calling the wrong tool), max_iterations stops it after N cycles. Without it, a confused agent could loop forever and burn your API budget. Typical value: 5–10.

Q6. What is the difference between short-term and long-term memory in agents?

Short-term memory: the current conversation stored in the LLM's context window. It disappears when the session ends (max ~200K tokens with GPT-4o). Long-term memory: past conversations stored in a vector database (Pinecone, ChromaDB). The agent retrieves relevant past info using similarity search.

Q7. What is handle_parsing_errors=True in AgentExecutor?

Sometimes the LLM returns text in an incorrect format (e.g., forgets to include 'Action:' prefix). Without this flag, the agent crashes with OutputParserException. With handle_parsing_errors=True, LangChain automatically retries with an error message, making the agent more robust.

Q8. Name three real-world applications of AI agents.

(1) Autonomous coding agents (like Claude Code / Devin) that write and debug code. (2) Customer support agents that look up orders, process refunds, send emails. (3) Research agents that search the web, summarize papers, generate reports. Other examples: travel booking, financial analysis, medical triage, HR automation.

Q9. What is LangChain Hub and why is hub.pull('hwchase17/react') used?

LangChain Hub is a repository of pre-built, community-vetted prompts. hwchase17/react is the canonical ReAct prompt that teaches the LLM to output responses in the exact Thought/Action/Action Input/Observation format that AgentExecutor expects. Using hub.pull() avoids writing this complex prompt

manually.

Q10. How would you add multiple tools to an agent?

Simply add all Tool objects to a Python list and pass it to `create_react_agent()` and `AgentExecutor()`. The LLM reads all tool descriptions and decides which one to call based on the question. Example: `tools = [search_tool, calculator_tool, weather_tool]`. The agent can chain multiple tools in a single task.

Section 10 · Quick Revision Cheatsheet

Core Formulas

```
■ python
# Compound Interest Formula
A = P * (1 + r/n)^(n*t)
# Where: P = Principal, r = annual rate (decimal),
# n = compounding per year, t = years
# A = final amount
# Simple version (annually compounded):
A = P * (1 + r)^t
# Example: Rs.50,000 at 8% for 5 years (annually compounded)
A = 50000 * (1.08)^5 = 73,466.40
Interest = A - P = 23,466.40
```

One-Page Agent Boilerplate

```
■ python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain import hub
load_dotenv()

def my_tool(query: str) -> str:
    return f'Result for: {query}' # replace with real logic

tools = [Tool(name='MyTool', func=my_tool,
description='Describe when to use this tool.')]

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
prompt = hub.pull('hwchase17/react')
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
exe = AgentExecutor(agent=agent, tools=tools, verbose=True,
max_iterations=5, handle_parsing_errors=True)

result = exe.invoke({'input': 'Your question here'})
print(result['output'])
```

Key Terms Flash Cards

Agent	AI that plans → uses tools → takes actions autonomously
ReAct	Reason + Act loop — how agents think step by step
Tool	Python function wrapped with name + description for the LLM
AgentExecutor	Runtime that manages the Reason→Act→Observe loop
LLM	The brain — large language model doing the reasoning

Memory	Short-term (context) or long-term (vector DB) storage
Prompt	Instructions that tell the LLM HOW to behave as an agent
max_iterations	Safety limit on how many loops the agent can take
verbose=True	Print every thought and action for debugging
hub.pull()	Download pre-built prompts from LangChain Hub

Free Alternatives to OpenAI API

Provider	Model	How to get free access	LangChain class
Groq	Llama 3.1 70B	console.groq.com → free tier (fast!)	ChatGroq
Google	Gemini 1.5 Flash	aistudio.google.com → API key → free	ChatGoogleGenerativeAI
Ollama	Llama 3 / Mistral	ollama.ai → runs 100% locally, free	ChatOllama
Anthropic	Claude Haiku	console.anthropic.com → \$5 free credits	ChatAnthropic

You have completed Day 22 · Introduction to AI Agents ■

Next: [Day 23 — Multi-Agent Systems & AutoGen](#)